

UNITÉ 7 · SEMAINES 10–12

Threads et concurrence

📅 Semaines 10–12 · dates à confirmer </> Laboratoire TD6 (corrigé + défi IA) ~ 5 animations

LABORATOIRE TD6 · CORRIGÉ

Threads et concurrence

Plusieurs threads partagent la mémoire d'un processus: c'est rapide et pratique, et c'est aussi la source des bogues les plus difficiles de la programmation. Ce laboratoire les provoque exprès et les corrige avec des mutex et des sémaphores.

✓ **Devoir : rapport du TP**

Rendez ce TP avec un court rapport en LaTeX (le strict nécessaire). Ce qu'il doit couvrir, le modèle, un exemple et le guide de Prism sont dans l'onglet Devoir. IA générative autorisée, avec déclaration.

EXERCICES

- | | | | |
|---|--|---|--|
| 1 | Créer et joindre des threads | 4 | Le dîner des philosophes |
| 2 | Vendeurs de billets: la course et deux solutions | 5 | Exploration: coût d'un thread, Peterson sur un x86 et le futex (nouveau) |
| 3 | Producteurs et consommateurs | | |

TD6 · 1

● Corrigé · testé

Créer et joindre des threads

Énoncé. Créez 8 threads avec `pthread_create` qui exécutent `start` et affichent le pid du processus (passé en argument). Le thread principal les attend tous avec `pthread_join`. Ajoutez une variable globale et une variable locale à `start`, incrémentez-les et affichez leurs adresses et leurs valeurs.

Solution



C tds/td6/threads.c

```

/*
 * TD6 - exercise 1: create 8 threads, wait for them, and compare a global
 and a local variable.
 * All threads share the global `shared` (same address) but each has its own
 `local` (on its
 * own stack): the addresses printed show it. `shared++` is a race: see
 exercise 2.
 * Build: gcc -pthread (the Makefile already does it).
 */
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define NTHREADS 8

static int shared = 0;

static void *start(void *arg)
{
    pid_t pid = (pid_t)(intptr_t)arg;      /* the pid travels inside the
void * */
    int local = 0;

    shared++;
    local++;
    printf("thread %lu of process %d: shared[%p] = %d, local[%p] = %d\n",
        (unsigned long)pthread_self() % 100000, pid, (void *)&shared,
shared, (void *)&local, local);
    return NULL;                          /* same as pthread_exit(NULL) */
}

int main(void)
{
    pthread_t tids[NTHREADS];
    pid_t pid = getpid();

    for (int i = 0; i < NTHREADS; i++) {
        int rc = pthread_create(&tids[i], NULL, start, (void *)
(intptr_t)pid);

```

```

        exit_if(rc != 0, "pthread_create"); /* pthread functions return the
error code */
    }
    for (int i = 0; i < NTHREADS; i++)
        exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");

    printf("process %d: all %d threads finished, shared = %d\n", pid,
NTHREADS, shared);
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1,
local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4,
local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5,
local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2,
local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3,
local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6,
local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7,
local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8,
local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8

```

Tous les threads affichent le **même pid** (ils forment un seul processus) et la **même adresse** pour `shared`, mais chaque `local` est à une adresse différente: chaque thread a sa propre pile. La valeur de `shared` sort dans le désordre et pourrait même se répéter: `shared++` n'est pas atomique.

Vendeurs de billets: la course et deux solutions

Énoncé. Pour une fête de la faculté, 8 étudiants (threads) vendent en parallèle les billets d'un compteur global: tant qu'il en reste, ils appellent `sched_yield`, en retirent un et comptent leur vente; à la fin, ils renvoient combien ils en ont vendu avec `pthread_exit` et le thread principal fait la somme. Essayez des quantités croissantes. Que se passe-t-il? Où est la section critique? Protégez-la avec un sémaphore et avec un mutex.

Solution

C tds/td6/tickets_race.c

```
static volatile int tickets = 1000;    /* volatile only prevents caching in a
register:

                                         it does NOT make the accesses
atomic */

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    while (tickets > 0) {                /* critical section starts here ...
*/
        sold++;
        sched_yield();                  /* do not touch this line
*/
        tickets--;                      /* ... and ends here
*/
    }
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;      /* same as pthread_exit((void *)sold)
*/
}
```

Sortie réelle

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

On vend plus de billets qu'il n'y en a et le compteur finit négatif. La section critique, c'est tout l'intervalle de `tickets > 0` à `tickets--` : entre le test et la soustraction, un autre thread peut vendre le dernier billet, et `tickets--` est lui-même lire-soustraire-écrire, trois étapes qui peuvent s'entrelacer.

C tds/td6/tickets_mutex.c

```
static int tickets = 10000;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    for (;;) {
        pthread_mutex_lock(&lock);
        if (tickets ≤ 0) {                               /* checked with the lock held */
            pthread_mutex_unlock(&lock);
            break;
        }
        sold++;
        sched_yield();
        tickets--;
        pthread_mutex_unlock(&lock);
    }
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;
}
```

Sortie réelle

```
$ ./tickets_mutex 100000 | tail -3
[3] I sold 11577 tickets
[2] I sold 11338 tickets
100000 tickets sold out of 100000 (tickets = 0)
```

C tds/td6/tickets_sem.c

```
static sem_t available;

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    while (sem_trywait(&available) == 0) {    /* took one ticket */
        sold++;
        sched_yield();
    }
    exit_if(errno != EAGAIN, "sem_trywait");    /* EAGAIN = no tickets left,
    anything else is a bug */
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *)(intptr_t)sold;
}
```

Sortie réelle

```
$ ./tickets_sem 100000 | tail -3
[6] I sold 14469 tickets
[2] I sold 11537 tickets
100000 tickets sold out of 100000 (tickets = 0)
```

Avec le **mutex**, le test et la soustraction se font verrou pris (et le test est répété à l'intérieur: la valeur lue avant de le prendre peut être périmée). Avec le **sémaphore**, la conception est encore meilleure: le sémaphore est le compteur de billets, et `sem_trywait` « vérifie et prend » en une seule opération indivisible.

Corrigée

La solution héritée avec mutex ne compilait pas (`nbtickets_venduss`) et lisait le compteur en dehors du verrou.

Animation A7.1 · Condition de course →

Producteurs et consommateurs

Énoncé. (a) Un producteur ajoute 1 à chaque case d'un tableau `A` et un consommateur fait la somme de toutes ses cases, `LOOP` fois chacun. Faites-en deux threads qui **alternent**, chacun avec un accès exclusif à tout le tableau. (b) Il y a maintenant 1 producteur et 4 consommateurs: chaque valeur produite doit être consommée exactement une fois. Quelles nouvelles sections critiques apparaissent?

Solution

C tds/td6/prodcons.c

```
/*
 * TD6 - exercise 3 (a): one producer and one consumer take turns on the
 * WHOLE array.
 * The producer adds 1 to every cell, then the consumer sums every cell; LOOP
 * rounds.
 * Expected sum: SIZE * (1 + 2 + ... + LOOP) = SIZE * LOOP * (LOOP + 1) / 2.
 *
 * Two semaphores implement the turn-taking ("baton passing"):
 * can_produce = 1 at start (the producer goes first), can_consume = 0.
 * The critical section is the traversal of the array: while one thread walks
 * it, the
 * other must not touch it. With the two semaphores, at most one of them is
 * inside.
 */
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define SIZE 10000
#define LOOP 10

static int A[SIZE];
static long sum = 0;
static sem_t can_produce, can_consume;

/* "Opaque" functions of the statement: they must not be modified. */
```

```
static int consume(int *value) { usleep(1); return *value; }
static void produce(int v, int *value) { usleep(1); *value += v; }

static void *producer(void *arg)
{
    (void)arg;
    for (int i = 0; i < LOOP; i++) {
        sem_wait(&can_produce);                /* my turn */
        for (int j = 0; j < SIZE; j++)
            produce(1, &A[j]);
        printf("producer: round %d written\n", i + 1);
        sem_post(&can_consume);                /* your turn */
    }
    return NULL;
}

static void *consumer(void *arg)
{
    (void)arg;
    for (int i = 0; i < LOOP; i++) {
        sem_wait(&can_consume);
        for (int j = 0; j < SIZE; j++)
            sum += consume(&A[j]);
        printf("consumer: sum after round %d = %ld\n", i + 1, sum);
        sem_post(&can_produce);
    }
    return NULL;
}

int main(void)
{
    pthread_t p, c;

    exit_if(sem_init(&can_produce, 0, 1) == -1 || sem_init(&can_consume, 0,
0) == -1, "sem_init");
    exit_if(pthread_create(&p, NULL, producer, NULL) != 0, "pthread_create");
    exit_if(pthread_create(&c, NULL, consumer, NULL) != 0, "pthread_create");
    exit_if(pthread_join(p, NULL) != 0 || pthread_join(c, NULL) != 0,
"pthread_join");

    long expected = (long)SIZE * LOOP * (LOOP + 1) / 2;
    printf("sum = %ld, expected %ld: %s\n", sum, expected, sum == expected ?
"OK" : "WRONG");
    sem_destroy(&can_produce);
    sem_destroy(&can_consume);
}
```

```
    return sum == expected ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Sortie réelle

```
$ ./prodcons | tail -4
consumer: sum after round 9 = 450000
producer: round 10 written
consumer: sum after round 10 = 550000
sum = 550000, expected 550000: OK
```

(a) Deux sémaphores se passent le tour comme un témoin de relais: `can_produce` commence à 1 et `can_consume` à 0. Chaque thread attend son sémaphore, parcourt le tableau et active l'autre. La somme attendue est $SIZE \times (1 + 2 + \dots + LOOP) = 10\,000 \times 55$.

C tds/td6/prodcons_multi.c

```
/*
 * TD6 - exercise 3 (b): one producer, N consumers, and a bounded buffer (the
 * classic of the
 * lecture). The producer puts ITEMS values in a circular buffer of BUF
 * cells; the consumers
 * take them out and add them to a shared sum. Every value must be consumed
 * exactly once.
 *
 *
 * empty = number of free cells (starts at BUF) - the producer waits on
 * it
 * full = number of used cells (starts at 0) - the consumers wait on
 * it
 * mutex = protects the `out` index shared by the consumers (new critical
 * section!)
 *
 * sum_lock protects the shared sum
 * To stop, the producer sends one "poison pill" (value -1) per consumer.
 * ./prodcons_multi [consumers] [items] (default 4 consumers, 100000
 * items)
 */
#include <pthread.h>
#include <semaphore.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"
```

```
#define BUF 16
#define POISON (-1)

static int buffer[BUF];
static int in = 0, out = 0;           /* in: producer only; out: shared
by consumers */
static sem_t empty, full;
static pthread_mutex_t out_lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t sum_lock = PTHREAD_MUTEX_INITIALIZER;
static long long sum = 0;
static int nconsumers = 4, items = 100000;

static void *producer(void *arg)
{
    (void)arg;
    for (int v = 1; v ≤ items + nconsumers; v++) {
        int value = v ≤ items ? v : POISON;
        sem_wait(&empty);           /* wait for a free cell */
        buffer[in] = value;         /* only one producer: no lock
needed for `in` */
        in = (in + 1) % BUF;
        sem_post(&full);           /* one more item available */
    }
    return NULL;
}

static void *consumer(void *arg)
{
    long mine = 0, count = 0;
    for (;;) {
        sem_wait(&full);
        pthread_mutex_lock(&out_lock); /* two consumers must not take
the same cell */
        int value = buffer[out];
        out = (out + 1) % BUF;
        pthread_mutex_unlock(&out_lock);
        sem_post(&empty);
        if (value == POISON)
            break;
        mine += value;
        count++;
    }
    pthread_mutex_lock(&sum_lock); /* add the partial sum once, not
per item */
    sum += mine;
}
```

```
pthread_mutex_unlock(&sum_lock);
printf("consumer %d: %ld items\n", (int)(intptr_t)arg, count);
return NULL;
}

int main(int argc, char *argv[])
{
    if (argc > 1) nconsumers = atoi(argv[1]);
    if (argc > 2) items = atoi(argv[2]);
    pthread_t prod, cons[nconsumers];

    exit_if(sem_init(&empty, 0, BUF) == -1 || sem_init(&full, 0, 0) == -1,
"sem_init");
    exit_if(pthread_create(&prod, NULL, producer, NULL) != 0,
"pthread_create");
    for (int i = 0; i < nconsumers; i++)
        exit_if(pthread_create(&cons[i], NULL, consumer, (void *) (intptr_t)i)
!= 0, "pthread_create");
    exit_if(pthread_join(prod, NULL) != 0, "pthread_join");
    for (int i = 0; i < nconsumers; i++)
        exit_if(pthread_join(cons[i], NULL) != 0, "pthread_join");

    long long expected = (long long)items * (items + 1) / 2;
    printf("sum = %lld, expected %lld: %s\n", sum, expected, sum == expected
? "OK" : "WRONG");
    return sum == expected ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Sortie réelle

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

(b) Le problème classique du tampon borné: `empty` compte les cases libres, `full` les cases occupées. Avec plusieurs consommateurs apparaît une nouvelle section critique: l'indice `out`, protégé par un mutex (sinon deux consommateurs prendraient la même case). La somme de chaque consommateur est ajoutée une seule fois, sous un autre mutex. Pour finir, le producteur envoie une « pilule empoisonnée » (-1) par consommateur.



Cohérence corrigée

L'énoncé parlait de `producteur / consommateur` et les solutions héritées de `lecteur / écrivain` ; l'une d'elles ne respectait pas « accès exclusif à tout le tableau » et celle à plusieurs consommateurs faisait de l'attente active avec `sem_trywait` + `usleep`. L'énoncé, le code et les noms concordent maintenant.

TD6 · 4

● Corrigé · testé (nouveau)

Le dîner des philosophes

Énoncé. 8 philosophes sont assis à une table ronde avec une baguette entre chaque paire; pour manger, chacun a besoin de la baguette de droite et de celle de gauche (un mutex par baguette). Tous prennent d'abord celle de droite, puis celle de gauche, et répètent penser–prendre–manger–relâcher. Que se passe-t-il? Proposez une modification qui l'évite.

Solution

C tds/td6/philosophers.c

```
/*
 * TD6 - exercise 4: the dining philosophers. N philosophers, N chopsticks
 * (one mutex each).
 * Every philosopher takes the RIGHT chopstick, then the LEFT one.
 * DEMO_DEADLOCK (Makefile target philosophers_deadlock): everybody may
 * hold the right
 * chopstick and wait forever for the left one: circular wait = deadlock. A
 * pause between
 * the two lock operations makes it happen almost every time.
 * default (solution): break the circular wait by ordering the resources -
 * every
 * philosopher takes the chopstick with the SMALLER number first. The last
 * philosopher
 * (whose right chopstick is N-1 and left is 0) therefore starts with the
 * left one.
 * ./philosophers [meals] (default 5 meals each)
 */
#include <pthread.h>
```

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define N 8

static pthread_mutex_t chopstick[N];
static int meals = 5;

static void think(int id)
{
    printf("%2d: thinking\n", id);
    usleep((useconds_t)(random() % 2000));
    printf("%2d: hungry\n", id);
}

static void eat(int id)
{
    printf("%2d: eating\n", id);
    usleep((useconds_t)(random() % 2000));
    printf("%2d: full\n", id);
}

static void *philosopher(void *arg)
{
    int id = (int)(intptr_t)arg;
    int right = id, left = (id + 1) % N;
#ifdef DEMO_DEADLOCK
    int first = right, second = left;
#else
    int first = right < left ? right : left;    /* global order on the
chopsticks */
    int second = right < left ? left : right;
#endif
    for (int m = 0; m < meals; m++) {
        think(id);
        pthread_mutex_lock(&chopstick[first]);
#ifdef DEMO_DEADLOCK
        usleep(20000);    /* everybody grabs the
first one... */
#endif
        pthread_mutex_lock(&chopstick[second]);
```

```

        eat(id);
        pthread_mutex_unlock(&chopstick[second]);
        pthread_mutex_unlock(&chopstick[first]);
    }
    printf("%2d: done after %d meals\n", id, meals);
    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t tids[N];

    if (argc > 1)
        meals = atoi(argv[1]);
    setvbuf(stdout, NULL, _IOLBF, 0);    /* line by line, even into a pipe:
see who is stuck */
    srand(2026);
    for (int i = 0; i < N; i++)
        exit_if(pthread_mutex_init(&chopstick[i], NULL) != 0,
"pthread_mutex_init");
    for (int i = 0; i < N; i++)
        exit_if(pthread_create(&tids[i], NULL, philosopher, (void *)
(intptr_t)i) != 0, "pthread_create");
    for (int i = 0; i < N; i++)
        exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
    printf("dinner finished: %d philosophers x %d meals\n", N, meals);
    return EXIT_SUCCESS;
}

```

Sortie réelle

```

$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status:
${PIPESTATUS[0]} (124 = still blocked after 3 s)"
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)

```

Sortie réelle

```
$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals
```

Si tout le monde prend d'abord celle de droite, il suffit que les 8 le fassent en même temps pour que chacun attende la baguette que tient son voisin : **interblocage**. La sortie réelle montre les 8 « affamés » et le programme bloqué jusqu'à ce que **timeout** le tue. Les quatre conditions de Coffman sont réunies (exclusion mutuelle, détention et attente, pas de réquisition et attente circulaire). La solution rompt l'**attente circulaire** : les baguettes sont numérotées et chaque philosophe prend d'abord celle de plus petit numéro ; le dernier philosophe prend donc d'abord celle de gauche et le cycle ne peut pas se fermer.

[Animation A7.3 · Interblocage →](#)

TD6 · 5

● Nouveau · corrigé · testé

Exploration: coût d'un thread, Peterson sur un x86 et le futex

Énoncé. Trois mesures pour vérifier (ou réfuter) trois affirmations des diapositives: (1) **create_cost** : créer un thread coûte-t-il moins cher que créer un processus? (diapositive 136). (2) **peterson** : l'algorithme de Peterson «est correct» (diapositive 152)... sur un vrai processeur? Comparez avec **peterson_fence**. (3) **lock_cost** sous **strace -c** : combien d'appels système un mutex fait-il avec et sans contention? (diapositives 160–162).

Solution et sorties réelles

C tds/td6/create_cost.c

```
/*
 * TD6 - exploration 1: how much does creating a thread cost compared with a
 * process? (original slide 136)
 * Creates and waits for N threads (pthread_create + pthread_join) and N
 * processes (fork + waitpid, the
 * child exits at once) and prints the average time of each.
 * ./create_cost [N]      default: 2000
 */
```

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void *nothing(void *arg)
{
    return arg;
}

int main(int argc, char *argv[])
{
    int n = argc > 1 ? atoi(argv[1]) : 2000;

    double t0 = now();
    for (int i = 0; i < n; i++) {
        pthread_t t;
        int rc = pthread_create(&t, NULL, nothing, NULL);
        exit_if(rc != 0, "pthread_create");
        exit_if(pthread_join(t, NULL) != 0, "pthread_join");
    }
    double thread_us = (now() - t0) * 1e6 / n;

    t0 = now();
    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            _exit(0);
        exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    }
    double fork_us = (now() - t0) * 1e6 / n;

    printf("%d threads:  %.1f us per pthread_create + pthread_join\n", n,

```

```
thread_us);
    printf("%d processes: %6.1f us per fork + waitpid\n", n, fork_us);
    printf("a process costs %.1f times a thread\n", fork_us / thread_us);
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

Un thread ne copie ni tables des pages ni descripteurs: il réserve seulement une pile et une structure de tâche. Sous Linux, les deux sont créés avec `clone` ; ce qui change, c'est ce qui est partagé.

C tds/td6/peterson.c

```
/*
 * TD6 - exploration 2: Peterson's algorithm on a real multicore x86
 (original slides 152 and 154).
 * Two threads increment a shared counter N times each, protecting the
 increment with Peterson's lock.
 * The algorithm is correct on paper, but it "assumes memory accesses are
 sequentially ordered per thread"
 * (slide 154). Without a fence, the x86 store buffer lets each thread read
 the other's flag BEFORE its own
 * store is visible: both enter the critical section and increments are lost.
 With a full fence
 * (mfence, via __atomic_thread_fence) the algorithm works. Compiled twice:
 peterson (no fence) and
 * peterson_fence (-DFENCE).
 * ./peterson [N]      default: 20000000
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

static volatile int flag[2];
static volatile int turn;
static volatile long counter;
static long n;
```

```
static void *worker(void *arg)
{
    int me = (int)(long)arg, other = 1 - me;
    for (long i = 0; i < n; i++) {
        flag[me] = 1;
        turn = other;
#ifdef FENCE
        __atomic_thread_fence(__ATOMIC_SEQ_CST);    /* the stores must be
visible before the loads */
#endif
        while (flag[other] && turn == other)
            ;                                       /* busy waiting */
        counter = counter + 1;                     /* critical section */
        flag[me] = 0;
    }
    return NULL;
}

int main(int argc, char *argv[])
{
    n = argc > 1 ? atol(argv[1]) : 2000000;
    pthread_t t[2];
    for (long i = 0; i < 2; i++)
        exit_if(pthread_create(&t[i], NULL, worker, (void *)i) != 0,
"pthread_create");
    for (int i = 0; i < 2; i++)
        exit_if(pthread_join(t[i], NULL) != 0, "pthread_join");
#ifdef FENCE
    const char *variant = "with fence";
#else
    const char *variant = "no fence";
#endif
    printf("Peterson %-10s: counter = %ld, expected %ld, lost %ld\n",
variant, counter, 2 * n, 2 * n - counter);
    return EXIT_SUCCESS;
}
```

Sortie réelle

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do
./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

Sans barrière, certaines exécutions perdent des incréments: les deux threads étaient en même temps dans la section critique. Le x86 laisse chaque écriture dans un **tampon** et permet à un thread de lire la note de l'autre avant que la sienne soit visible (le modèle TSO). `__atomic_thread_fence` (une instruction `mfence`) l'empêche. C'est ce dont avertit la diapositive 154 et ce que vérifie l'animation A7.5, état par état.

C tds/td6/lock_cost.c

```
static void *worker(void *arg)
{
    for (long i = 0; i < n; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return arg;
}
```

Sortie réelle

```
$ ./lock_cost 1; ./lock_cost 8
1 thread(s) x 1000000 lock/unlock: counter = 1000000, 28.2 ns per lock/unlock
8 thread(s) x 1000000 lock/unlock: counter = 8000000, 184.2 ns per
lock/unlock
```

Sortie réelle

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e
trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00 0.000000 0 1 futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00 0.223730 27 8013 3116 futex
```

Sans contention, 100 000 `lock / unlock` ne font **aucun** appel à `futex` (le seul est celui du `join`) : un mutex libre se prend avec une instruction atomique en espace utilisateur. Ce n'est que lorsqu'un thread doit attendre qu'il entre dans le noyau. C'est l'« attente active limitée » de la diapositive 160, rendue réelle par le *futex* de Linux.

Le voir vérifié → A7.5 le problème du lait

Liste de vérification

Défi avec l'IA générative · Philosophes: détecter l'interblocage avec un graphe ^

Objectif. Instrumentez ``philosophers.c`` pour enregistrer qui attend qui et détecter automatiquement le cycle du graphe d'attente ; comparez deux solutions (ordre global et sémaphore « serveur »).

Prompt de départ. Utilisez-le dans l'outil de votre choix (Claude, ChatGPT, Gemini, Copilot, Codex...) avec la version *corrigée* du laboratoire comme contexte. Demandez d'abord un plan et relisez chaque étape avant de l'accepter.

Prompt

J'ai le dîner des philosophes avec des mutex [coller `philosophers.c`]. Aide-moi à :
 (1) ajouter un thread de surveillance qui, toutes les 100 ms, construit le graphe « le philosophe *i* attend la baguette tenue par *j* » et détecte un cycle ; (2)
 implémenter la solution du serveur (un sémaphore qui laisse s'asseoir *N-1* philosophes) et la comparer à celle de l'ordre global en repas par seconde ; (3)
 relier chaque solution à la condition de Coffman qu'elle rompt. Avant de coder, demande-moi laquelle des 4 conditions chacune rompt.

Comment vérifier le résultat

- Le thread de surveillance détecte le cycle dans ``philosophers_deadlock`` et affiche les 8 philosophes du cycle.
- Les deux solutions se terminent toujours (répétez 20 fois).
- Vos chiffres de repas par seconde viennent d'une mesure, pas de l'IA.

Déclarez votre usage de l'IA : outil, prompts pertinents, ce que vous avez vérifié vous-même et ce que vous avez corrigé. L'important est que vous puissiez expliquer et vérifier chaque décision.